You said:

Unique modern name

ChatGPT said:

For a modern Bluetooth-synced expense-sharing app, I'd choose a name that's **short, memorable, premium, and brandable**.

Name	Brand feel
Nexora	Modern, premium, futuristic
Pactly	Friends, shared commitments
Tabora	Bills, groups, effortless splitting
Veylo	Minimal, stylish, startup-like
Klyro	Sharp, tech-forward
Tandri	Together, trips, companionship
Oweo	Playful, expense-focused
Mavio	Friendly, premium, travel-oriented
Duoza	Social, energetic
Fynlo	Finance meets modern branding

My top 3 picks

- **Veylo**: Best for a premium minimalist app.
- **Pactly**: Best for group expense sharing.
- **Nexora**: Best for a broader lifestyle and travel brand.

My personal pick: VEYLO.

Tagline: Together, expenses made effortless.

If you want, I can suggest **10 even more distinctive names with the vibe of Linear, Revolut, Arc, and Notion**.

You said:

Now give full proper guide instruction

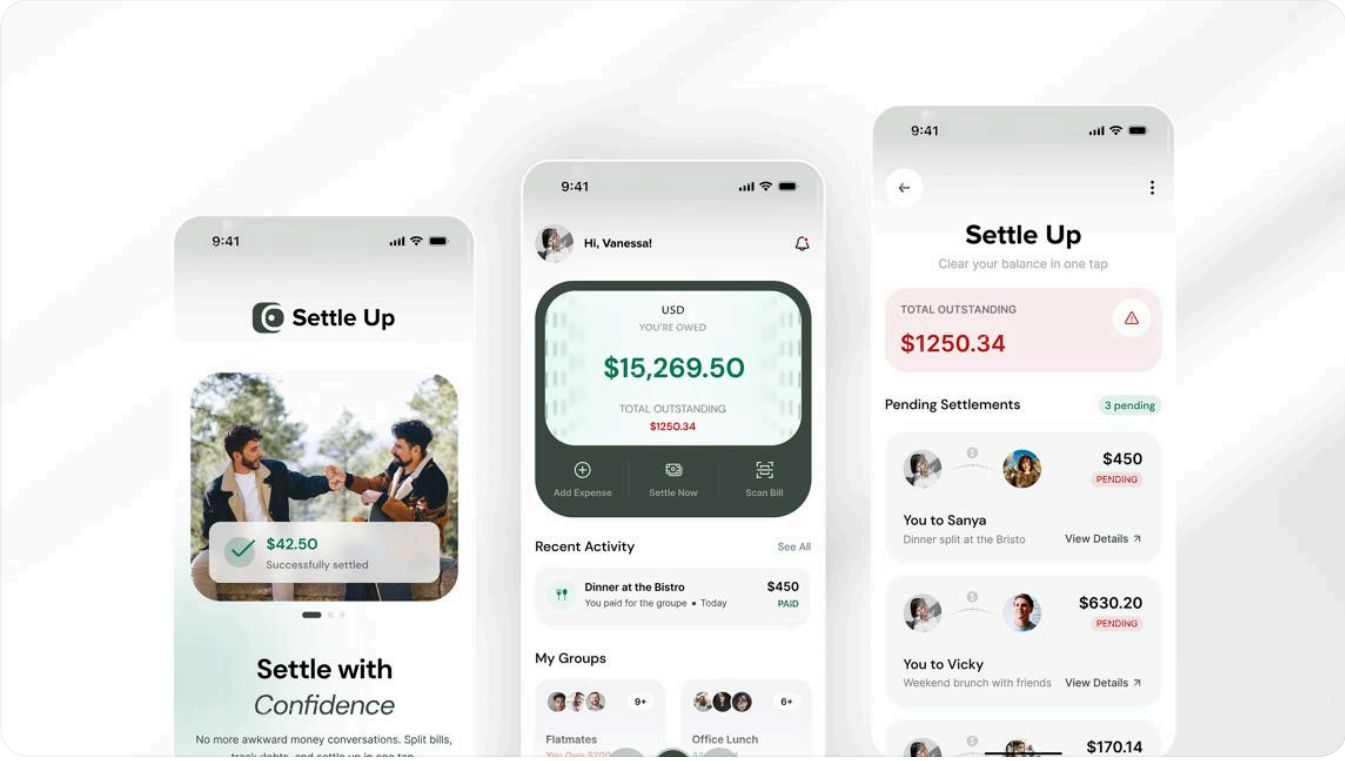
ChatGPT said:

Below is the complete product and development specification for **VEYLO**, your Kotlin Android application for group trips, café outings, shared expenses, and real-time Bluetooth synchronization.

The goal is to build a genuinely functional, installable APK with a premium modern UI, not just a static design prototype.

VEYLO

Product specification and AI development instructions



- **App name:** VEYLO
- **Tagline:** Together, expenses made effortless.
- **Platform:** Android
- **Language:** Kotlin
- **UI:** Jetpack Compose + Material 3
- **Database:** Room, offline-first
- **Connectivity:** Google Nearby Connections API
- **Backend:** None required for MVP
- **Authentication:** No mandatory login
- **Output:** Installable APK

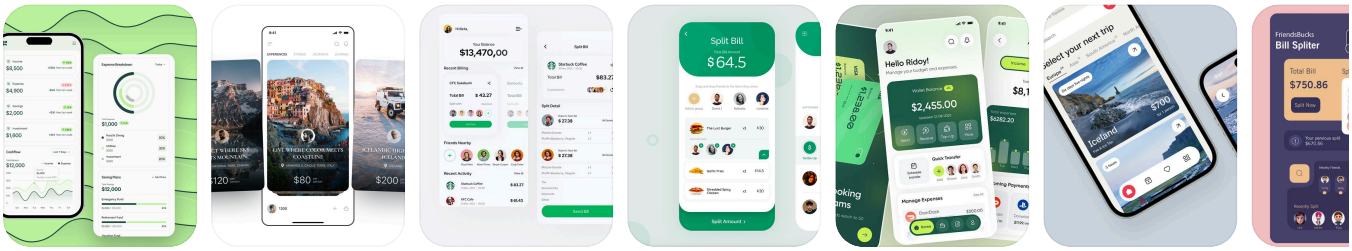
1. Product vision

VEYLO is a social expense-sharing application that makes group spending effortless.

Users can create trips, start café outings, add friends, record payments, split bills, and settle balances. All participants can synchronize expense records locally between Android devices using Bluetooth or supported nearby connectivity, even without internet.

The defining feature is **instant shared expense updates without requiring everyone to manually enter the same transaction**.

2. UI/UX design system



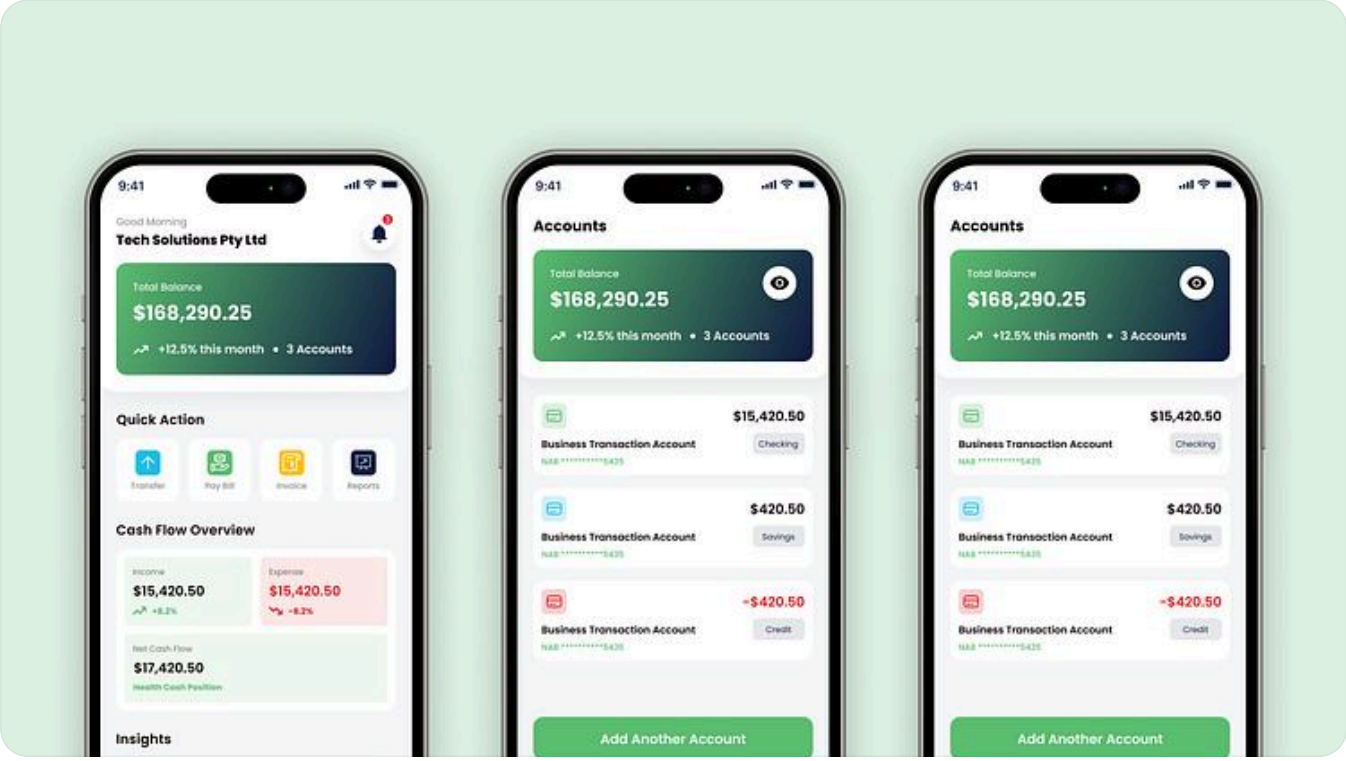
Visual identity

Element	Specification
Primary	Forest Green `#163D2A`
Accent	Electric Lime `#C8FF65`
Background	Warm Cream `#F7F7F2`
Surface	White `#FFFFFF`
Text	Deep Charcoal `#1C211D`
Typography	Manrope
Corners	20–28dp
Style	Premium editorial fintech
Theme	Light + Dark

Design requirements:

- Large expressive balance numbers.
- Beautiful trip cover images.
- Rounded cards and minimal borders.
- Subtle motion and haptic feedback.
- Consistent iconography using Material Symbols.
- Skeleton loading and polished empty states.
- Smooth bottom sheets for expense creation.
- No unnecessary gradients, clutter, or excessive colors.

3. Application navigation



Use four main navigation tabs:

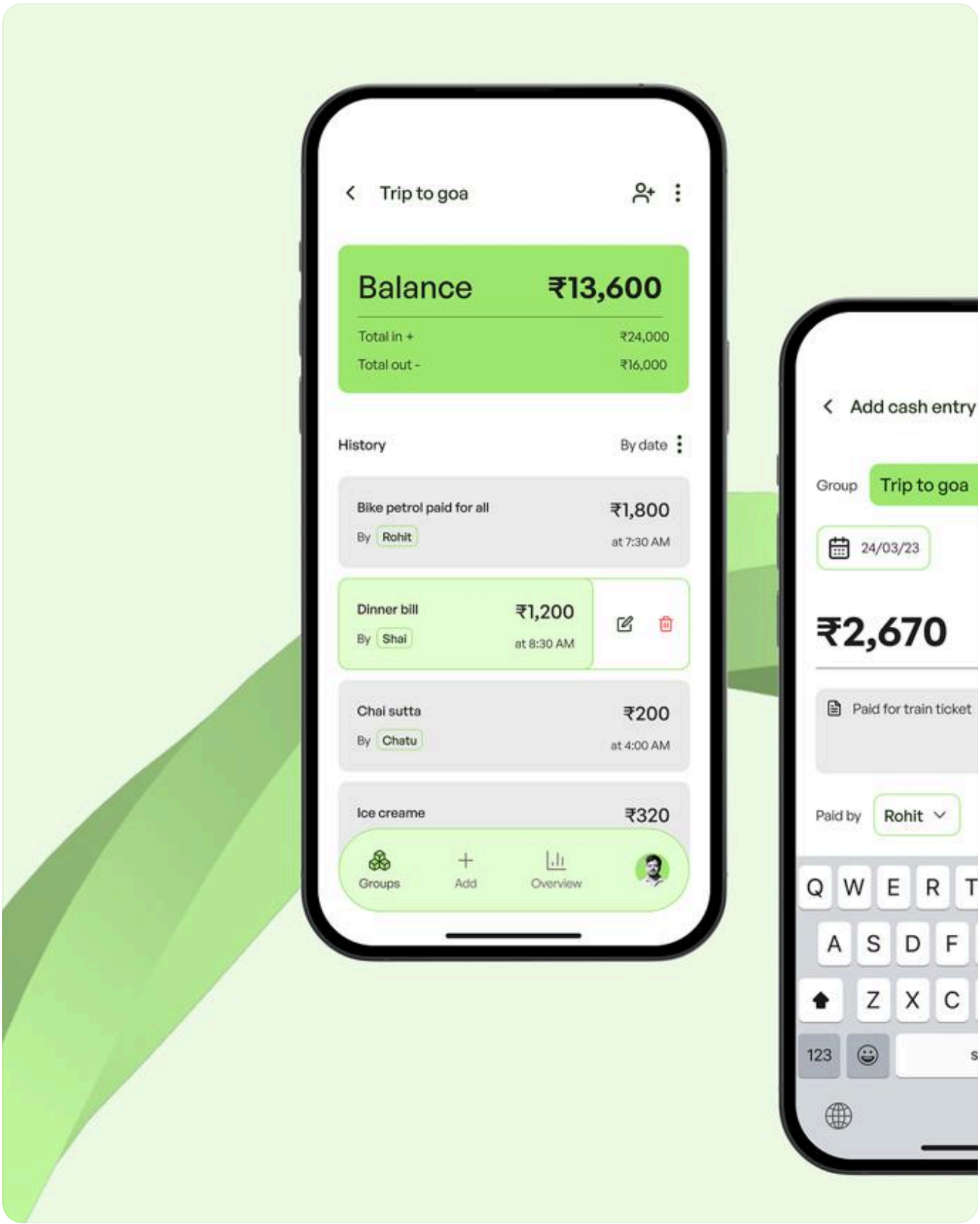
Tab	Purpose
Home	Overview, active trips, recent activity
Groups	Trips, outings, group management
Activity	Expense and settlement timeline
Profile	Preferences, app settings, backup

A floating + button should open a compact action menu:

- Add Expense
- Split Bill
- Create Trip
- Start Outing

4. Complete application screens

A. Home dashboard



Display:

- Greeting and profile avatar.

- Total amount you owe.
- Total amount owed to you.
- Net balance.
- Active trip or outing.
- Recent transactions.
- Bluetooth connection status.
- Quick action buttons.

Example:

```
VEYLO                          Profile

Good evening 👋

Your balance
₹1,250.00
You are owed

[ + Add Expense ]

Your Groups
[ Darjeeling Trip ]
[ Café Weekend ]

Recent Activity
Sayan paid ₹1,200
Rahul settled ₹300
```

B. Create trip

Fields:

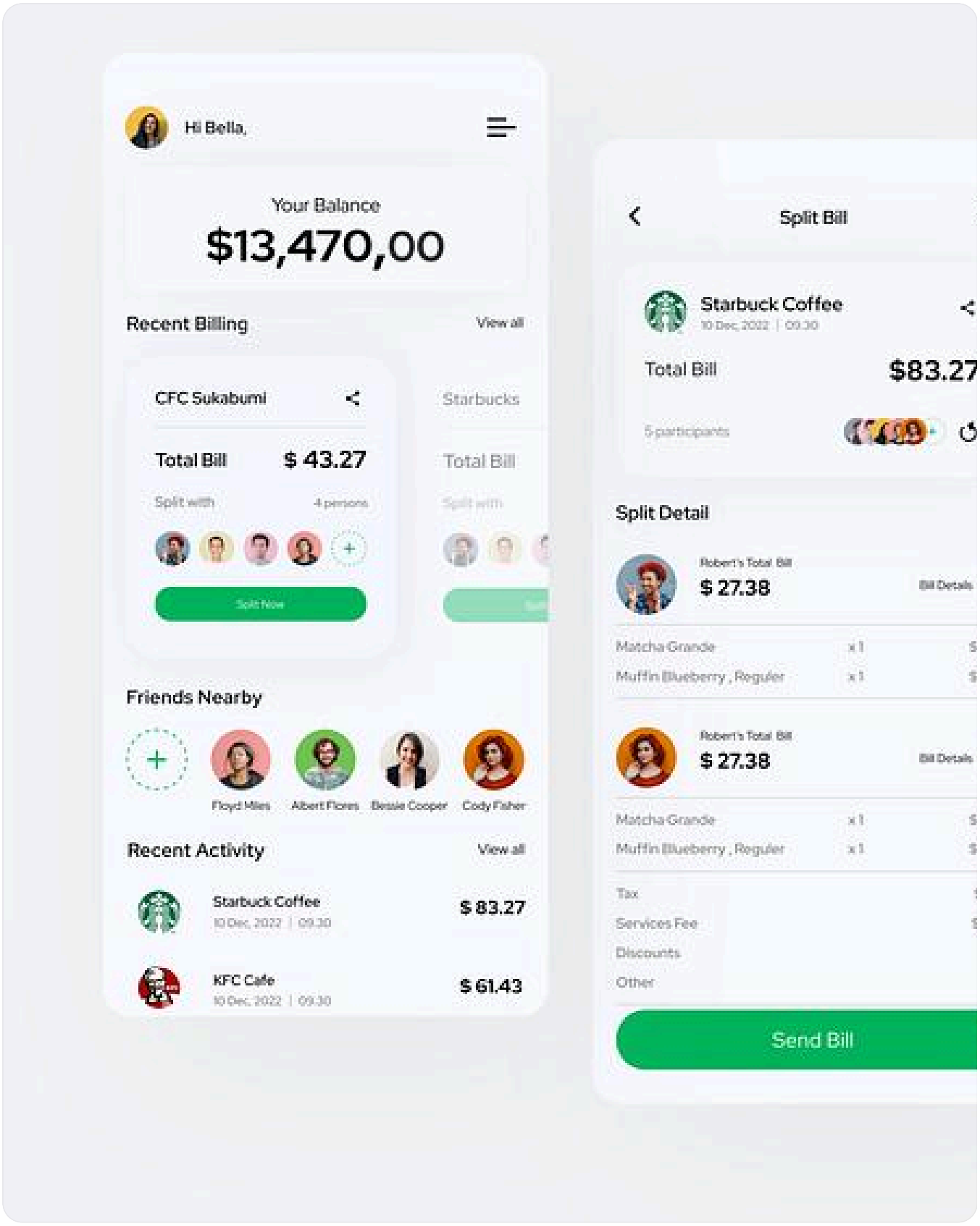
- Trip name.
- Destination.
- Start and end date.
- Trip cover image.
- Currency.
- Participants.

Features:

- Generate QR code to invite members.
- Discover nearby devices.
- Create a unique trip ID.
- Start Bluetooth group synchronization.

C. Outing mode

Designed for spontaneous café visits, restaurants, shopping, and weekend outings.

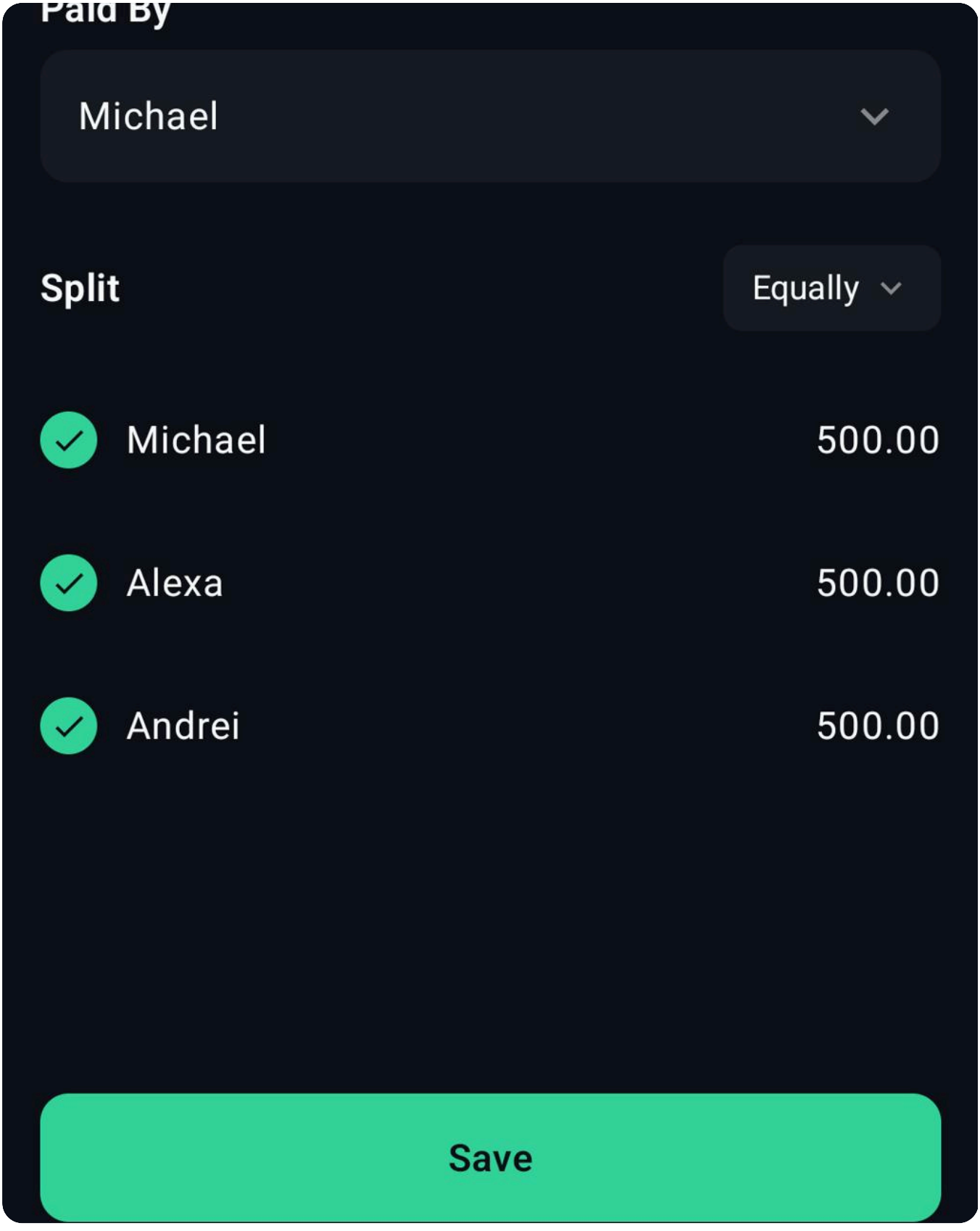


Features:

- Quick group creation.
- Add members using QR or Bluetooth.
- Enter total bill.
- Select who paid.
- Select participants.

- Calculate each member's share.
- Record tips and additional charges.
- Add receipt photo.

D. Expense creation



Expense fields:

Field	Example
Title	Café bill
Amount	₹1,200
Paid by	Sayan
Participants	4 friends
Category	Food
Split method	Equal
Date	Current date
Receipt	Optional

Split methods:

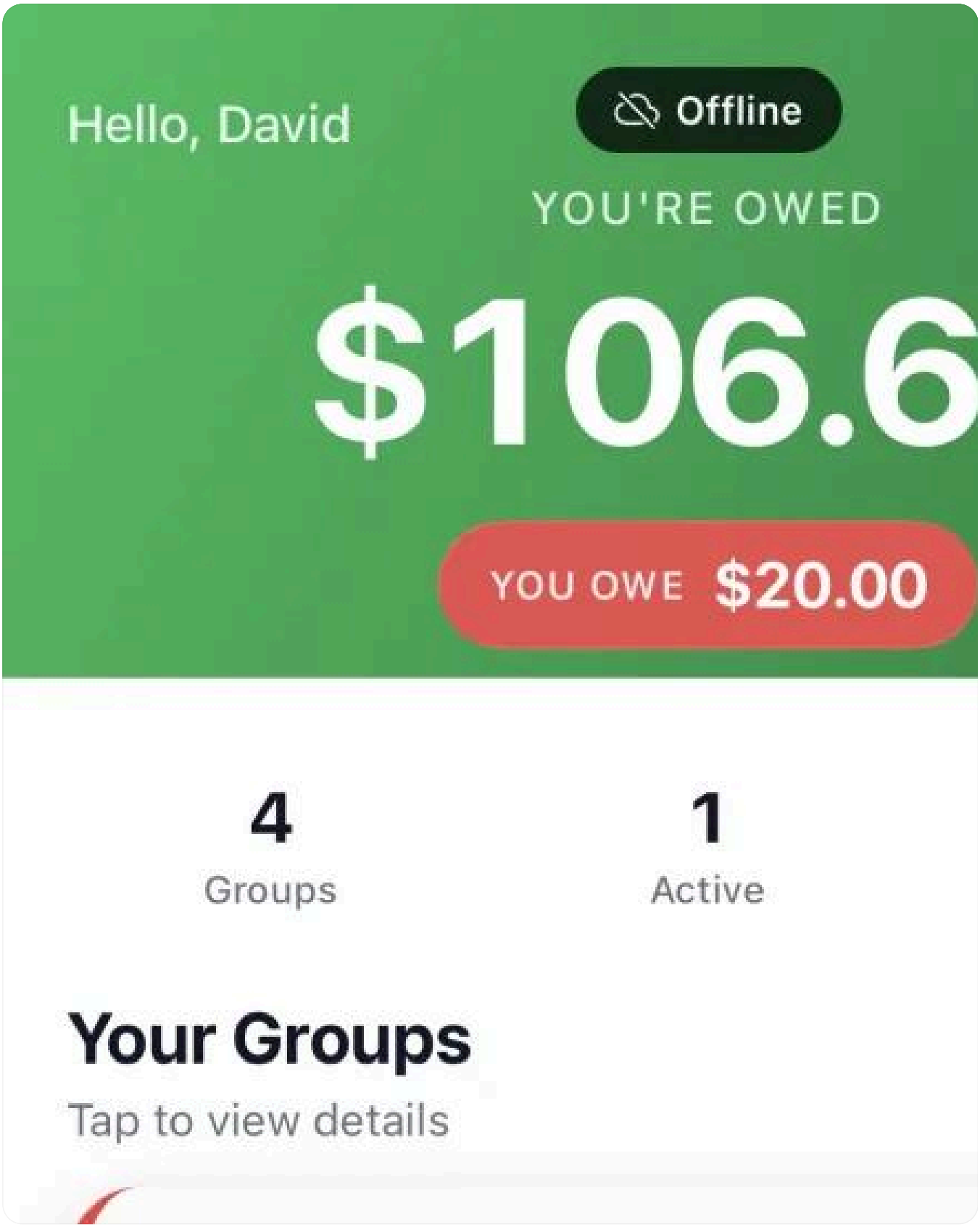
- Equal split.
- Custom amount.
- Percentage.
- Shares or weighted split.
- Exclude selected members.

E. Expense details

Display:

- Total expense.
- Paid by.
- Individual contribution/share.
- Participant list.
- Remaining balances.
- Receipt image.
- Sync status.
- Edit history.

F. Settlement



Features:

- Who owes whom.
- Minimum settlement transactions.
- Record settlement.

- Optional UPI payment link.
- Settlement history.

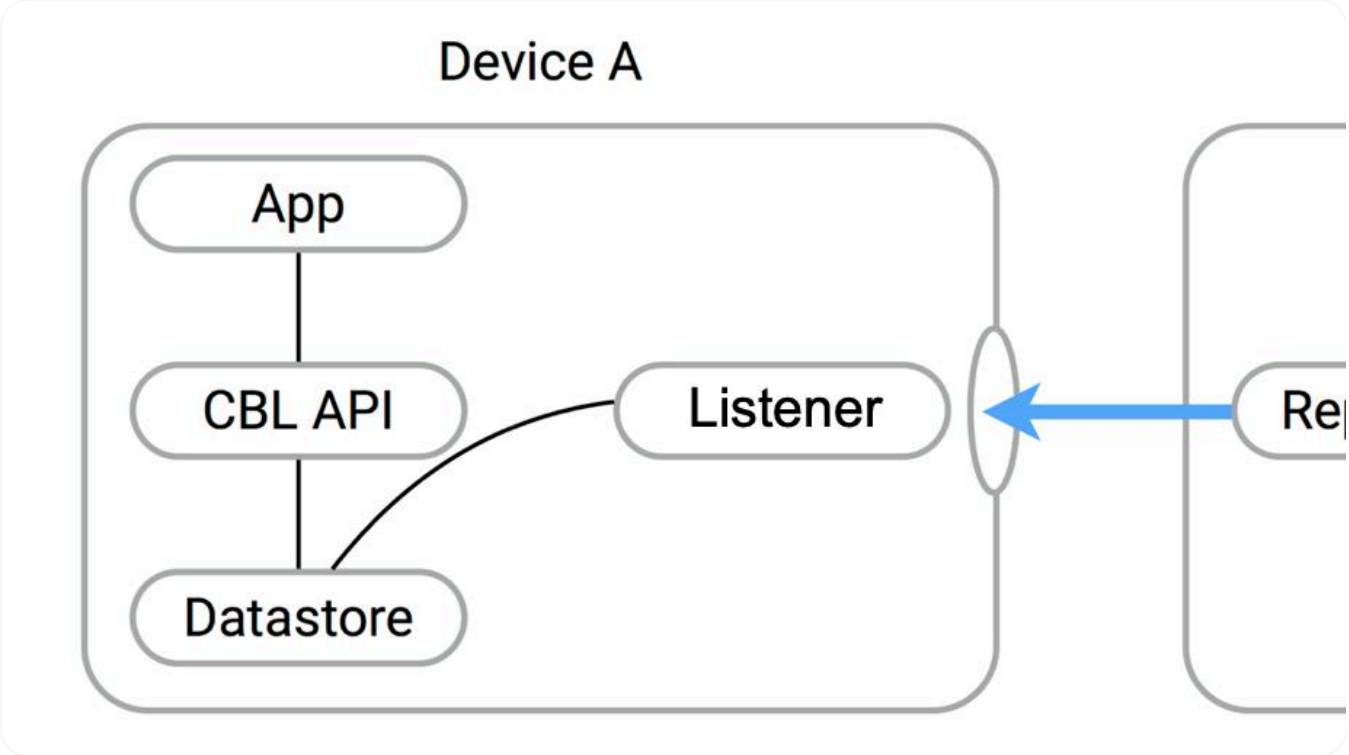
Example:

```
Settlement Summary

Rahul owes Sayan ₹300

[ Pay via UPI ]
[ Mark as Settled ]
```

5. Bluetooth synchronization architecture



Recommended technology:

Google Nearby Connections API

It supports nearby device discovery and data exchange using Bluetooth, BLE, and Wi-Fi depending on availability. It does not require an internet connection for nearby communication.

Connection workflow

```
Create Group
     ↓
Host Device Starts Advertising
     ↓
Other Devices Discover Host
     ↓
```

```
QR / Group Code Verification
     ↓
Connection Accepted
     ↓
Initial Ledger Synchronization
     ↓
Real-Time Expense Updates
```

Synchronization behavior

When a member creates an expense:

```
1. Create unique expense event.
2. Save event in Room database.
3. Broadcast event to connected members.
4. Host validates and records the event.
5. Host forwards event to all group members.
6. Each device updates its local database.
7. UI updates automatically using Kotlin Flow.
8. Show synced confirmation.
```

Important technical requirements

- Each event must have a globally unique ID.
- Prevent duplicate expenses.
- Keep an event history.
- Track synchronization acknowledgements.
- Retry failed deliveries.
- Synchronize missing events on reconnection.
- Handle multiple members recording expenses simultaneously.
- Maintain consistent balances across devices.
- Detect conflicting edits and resolve them deterministically.

Recommended MVP topology: One group host with multiple connected participants. The host maintains the authoritative group ledger while every phone stores a local copy. If the host disconnects, members can continue recording offline events and synchronize when the host returns.

Do not treat Bluetooth connection status as proof that an expense has synchronized. Track actual event acknowledgements.

6. Technical architecture

Recommended Android project structure

```
VEYLO/
├── app/
│   ├── data/
│   │   ├── local/
```

```
│   │   │   ├── VeyloDatabase.kt
│   │   │   ├── ExpenseDao.kt
│   │   │   └── GroupDao.kt
│   │   ├── repository/
│   │   │   ├── ExpenseRepository.kt
│   │   │   └── SyncRepository.kt
│   │   └── sync/
│   │       ├── NearbyManager.kt
│   │       ├── SyncEngine.kt
│   │       ├── EventProcessor.kt
│   │       └── ConflictResolver.kt
│   ├── domain/
│   │   ├── model/
│   │   ├── usecase/
│   │   └── settlement/
│   ├── ui/
│   │   ├── home/
│   │   ├── groups/
│   │   ├── outing/
│   │   ├── expense/
│   │   ├── settlement/
│   │   ├── activity/
│   │   ├── profile/
│   │   └── components/
│   ├── navigation/
│   └── MainActivity.kt
```

Technology stack

Component	Technology
Language	Kotlin
UI	Jetpack Compose
Design	Material 3
Database	Room
Architecture	MVVM
State	StateFlow
Async	Coroutines
Nearby communication	Nearby Connections
Serialization	Kotlin Serialization
QR	ZXing or ML Kit Barcode Scanning
Image loading	Coil
Dependency injection	Hilt

Testing	JUnit + Compose UI tests

7. Database design

Use Room for all local data.

Main entities

User

- userId
- displayName
- avatar
- deviceId

Group

- groupId
- groupName
- groupType
- destination
- createdAt
- hostMemberId

GroupMember

- memberId
- groupId
- displayName
- deviceId
- role
- joinedAt

Expense

- expenseId
- groupId
- title
- amount
- payerId
- category
- createdAt
- createdBy
- syncStatus

ExpenseSplit

- splitId
- expenseId
- memberId

- amountOwed

Settlement

- settlementId
- groupId
- fromMember
- toMember
- amount
- status
- createdAt

SyncEvent

- eventId
- groupId
- eventType
- payload
- originMemberId
- createdAt
- sequenceNumber

8. Security and privacy

- No unnecessary user permissions.
- No contact list access without explicit consent.
- Nearby devices must verify group membership.
- Use authenticated encrypted connections.
- Never trust expense events from unknown devices.
- Validate all amounts and participant IDs.
- Prevent unauthorized expense modification.
- Store no payment credentials.
- Provide a group leave and data deletion option.
- Keep personal information local by default.

For Bluetooth and nearby communication, implement the permission requirements applicable to the Android versions you support. Nearby Connections also requires its corresponding Google Play services availability.

9. Development roadmap

Phase 1 UI foundation

- Setup Android Studio project.
- Configure Kotlin and Compose.
- Implement design system.
- Build navigation.
- Create home dashboard and group screens.

Phase 2 Expense engine

- Room database.
- Group management.
- Expense creation.
- Split calculations.
- Settlement calculations.
- Activity timeline.

Phase 3 Nearby synchronization

- Nearby Connections integration.
- Advertising and discovery.
- Connection acceptance.
- Group pairing.
- Expense broadcasting.
- Ledger synchronization.
- Reconnection and conflict resolution.

Phase 4 Production polish

- UI animations.
- Dark mode.
- Receipt upload.
- QR invitations.
- UPI integration.
- Permission handling.
- Error states.
- Testing and bug fixes.

Phase 5 APK release

- Build debug APK.
- Test on multiple physical devices.
- Verify offline synchronization.
- Generate signed release APK.
- Install and demonstrate.

10. Master AI development prompt

Use this as the primary instruction for Cursor, Antigravity, Claude Code, or another AI coding agent.

Writing

PROJECT: VEYLO

Premium Offline Group Expense Sharing Android App

Act as a senior Android engineer, product designer, and mobile application architect.

Build VEYLO, a complete Android application using Kotlin, Jetpack Compose, Room, MVVM, and Google Nearby Connections API.

The application enables friends to create trips, café outings, and shared groups, record expenses, split bills, synchronize expense records between nearby Android devices without internet, and settle balances.

PRODUCT IDENTITY

Name: VEYLO
Tagline: Together, expenses made effortless.

Brand positioning:
Premium, modern, minimal, social, travel-oriented fintech application.

DESIGN REQUIREMENTS

Create a high-quality interface inspired by modern Dribbble fintech, travel, and expense-sharing applications.

Use:

- Forest Green #163D2A
- Electric Lime #C8FF65
- Warm Cream #F7F7F2
- White surfaces
- Manrope typography
- Rounded cards
- Premium whitespace
- Beautiful destination imagery
- Minimal icons
- Smooth transitions
- Haptic feedback
- Premium light and dark themes

Avoid:

- Generic outdated Android UI
- Excessive borders
- Crowded dashboards
- Unnecessary gradients
- Inconsistent spacing
- Placeholder-only screens
- Non-functional buttons

APPLICATION FEATURES

1. Home dashboard
2. Create and manage trips
3. Start spontaneous outings
4. Create and manage expense groups
5. QR-based member invitations
6. Nearby Bluetooth device discovery

7. Real-time shared expense updates
8. Equal and custom bill splitting
9. Expense categories
10. Receipt attachments
11. Individual member balances
12. Settlement optimization
13. Settlement history
14. Optional UPI payment deep links
15. Activity timeline
16. Offline-first operation
17. Reconnection and synchronization
18. Dark mode
19. Local profile management
20. APK generation

BLUETOOTH SYNCHRONIZATION

Use Google Nearby Connections API.

Implement:

- Group host advertising.
- Participant discovery.
- Connection requests.
- Group verification.
- Initial database synchronization.
- Real-time expense event broadcasting.
- Settlement event broadcasting.
- Event acknowledgement.
- Duplicate event prevention.
- Reconnection.
- Missing-event recovery.
- Concurrent expense conflict handling.
- Host-authoritative ledger.
- Local offline event queue.

Every expense must be saved locally before broadcasting.

The interface must distinguish:

- Local
- Pending synchronization
- Synchronizing
- Synced
- Failed

Never falsely display an expense as synchronized before receiving the required acknowledgement.

ARCHITECTURE

Use:

- Kotlin
- Jetpack Compose
- Material 3
- MVVM
- Room
- StateFlow
- Coroutines
- Hilt
- Nearby Connections
- Kotlin Serialization

Maintain separation between:

- UI
- Domain logic
- Database
- Synchronization
- Navigation

IMPLEMENTATION INSTRUCTIONS

Do not generate the entire application as a single unstructured file.

Create a maintainable modular project.

Start by:

1. Establishing the project structure.
2. Configuring dependencies.
3. Creating the design system.
4. Building navigation.
5. Implementing Room entities.
6. Implementing expense calculations.
7. Implementing settlement calculations.
8. Implementing Nearby Connections.
9. Implementing synchronization.
10. Implementing all UI screens.
11. Adding error handling.
12. Testing on multiple Android devices.
13. Building a functional APK.

QUALITY REQUIREMENTS

All primary buttons must work.

All expense calculations must be correct.

All data must persist across app restarts.

Bluetooth synchronization must work without internet access.

Handle disconnections and reconnect automatically when possible.

No mandatory login or backend.

Do not leave major features as mock implementations.

Do not claim synchronization works until it has been tested on physical devices.

FINAL DELIVERABLE

A complete Android Studio project for VEYLO.

Provide:

- Source code.
- Build configuration.
- Database implementation.
- Synchronization implementation.
- Functional screens.
- Test cases.
- Setup instructions.
- APK build instructions.
- Debug APK.
- Release APK signing instructions.

Prioritize functionality, reliability, excellent UX, and maintainable code.

11. Recommended next step

I would build VEYLO in this order:

1. Generate the complete premium UI.
2. Implement the expense engine and Room database.
3. Implement Bluetooth synchronization.
4. Test on two or more Android phones.
5. Generate the APK.

One design decision worth making now: use a premium cream-and-green interface with electric lime accents, and make the Bluetooth-connected group avatars a distinctive part of the home screen.

If you want to proceed directly to development, I can also prepare a **screen-by-screen implementation prompt with exact UI layouts, component specifications, animations, and functional acceptance tests** for your coding agent.